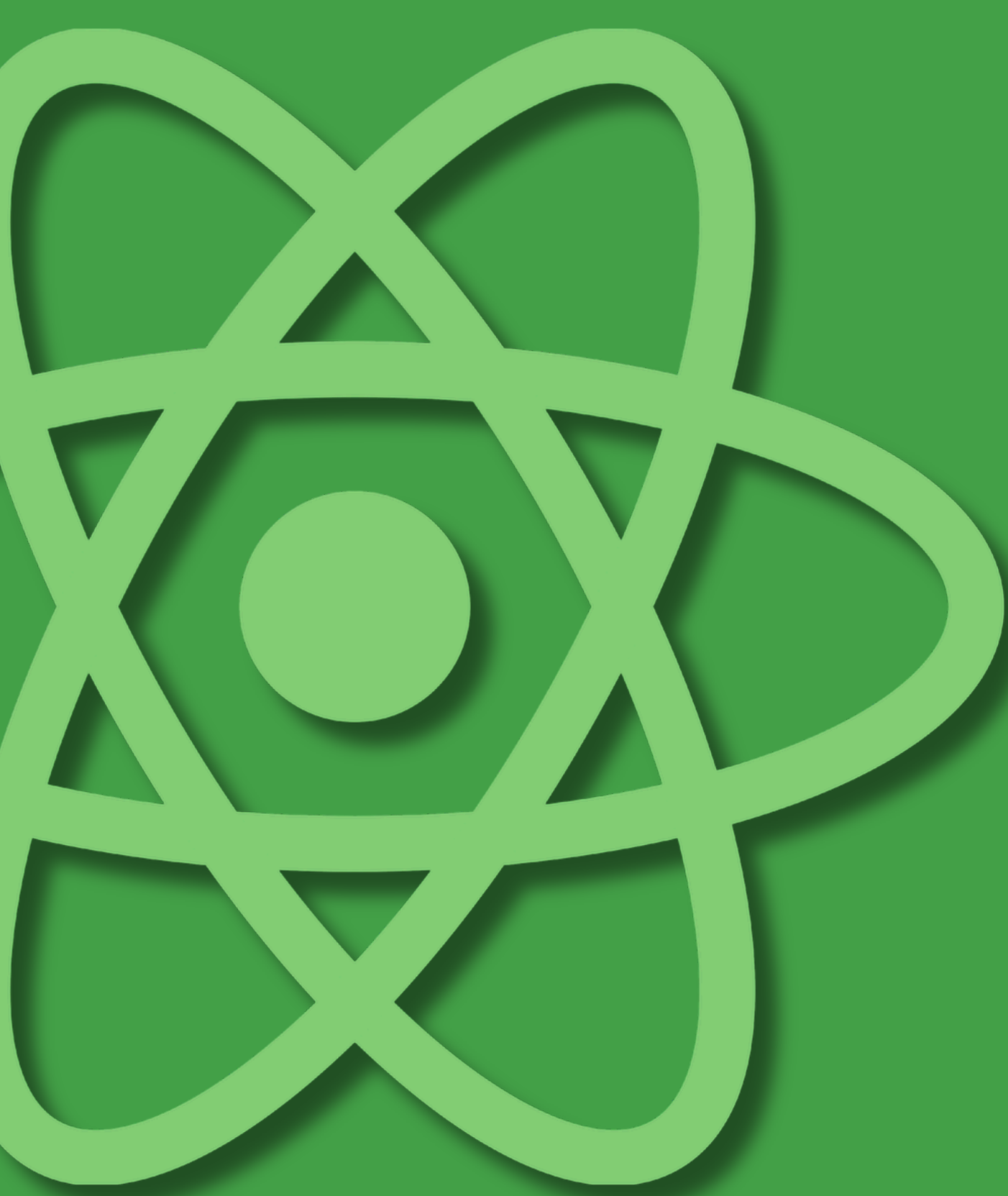




React.js

Docente: Matteo Valenzi

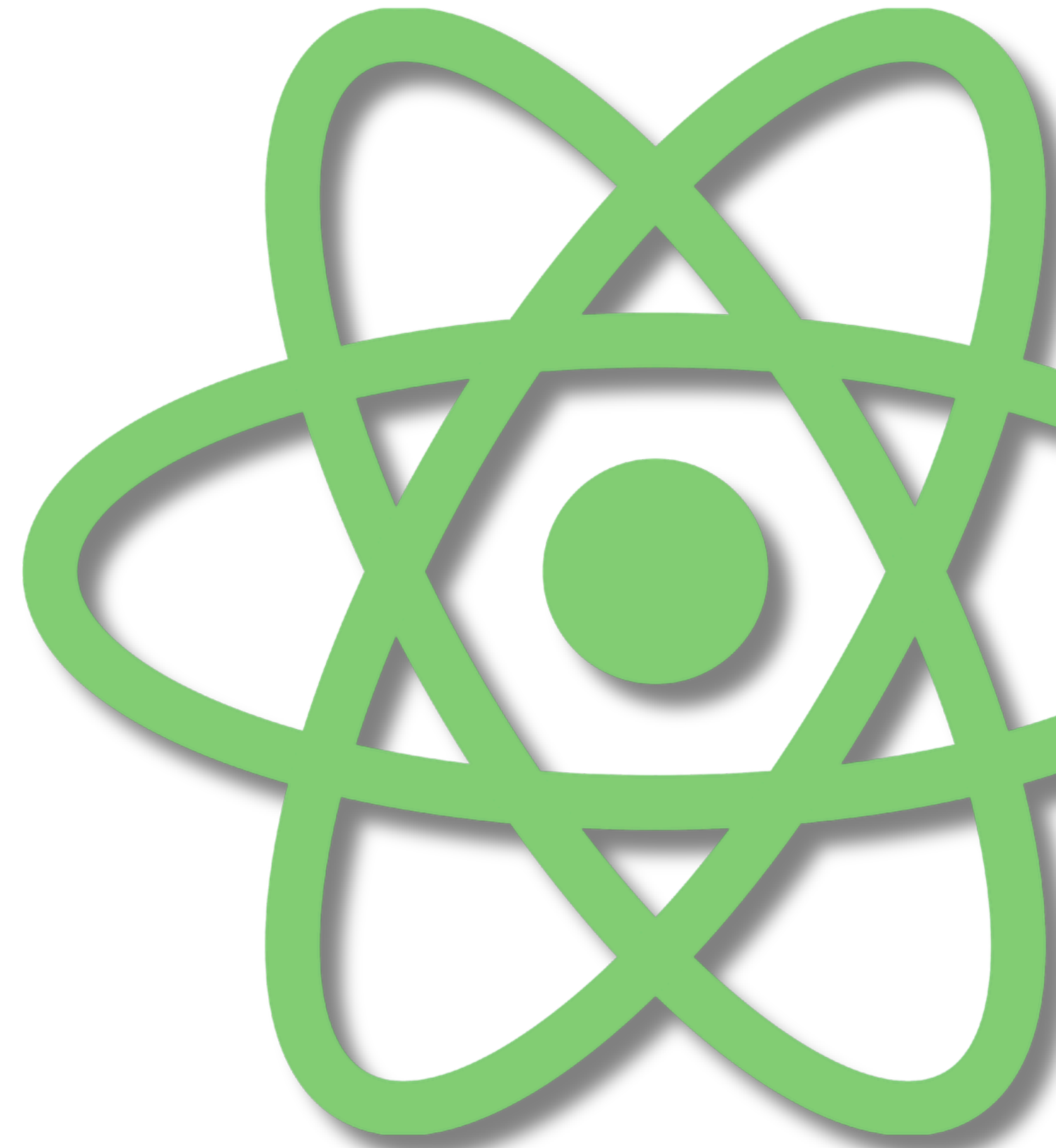
Setup e Introduzione

Benvenuti al Corso React

Questo corso vi guiderà attraverso lo sviluppo di applicazioni moderne con **React** e **TypeScript**

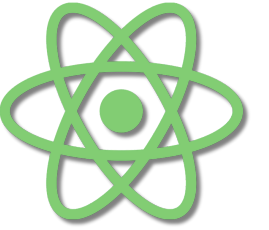
Obiettivi del corso:

- Comprendere la filosofia di React
- Sviluppare applicazioni scalabili e mantenibili
- Utilizzare TypeScript per un codice type-safe
- Gestire stato, routing e form in modo professionale



Setup e Introduzione

React: Filosofia e Principi



Filosofia di React:

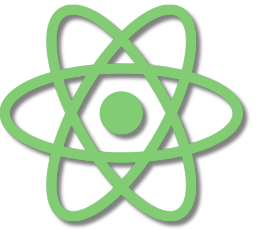
- **Dichiarativo**: descrivi "cosa" vuoi, non "come" ottenerlo
- **Component-based**: UI costruita da pezzi riutilizzabili
- **Learn once, write anywhere**: logica trasferibile

Principi fondamentali:

- **Composizione**: costruire UI complesse da componenti semplici
- **Unidirezionalità**: flusso dati top-down prevedibile
- **Immutabilità**: stato modificato tramite nuove copie
- **Rendering efficiente**: Virtual DOM e riconciliazione

Setup e Introduzione

Setup con Vite e TypeScript

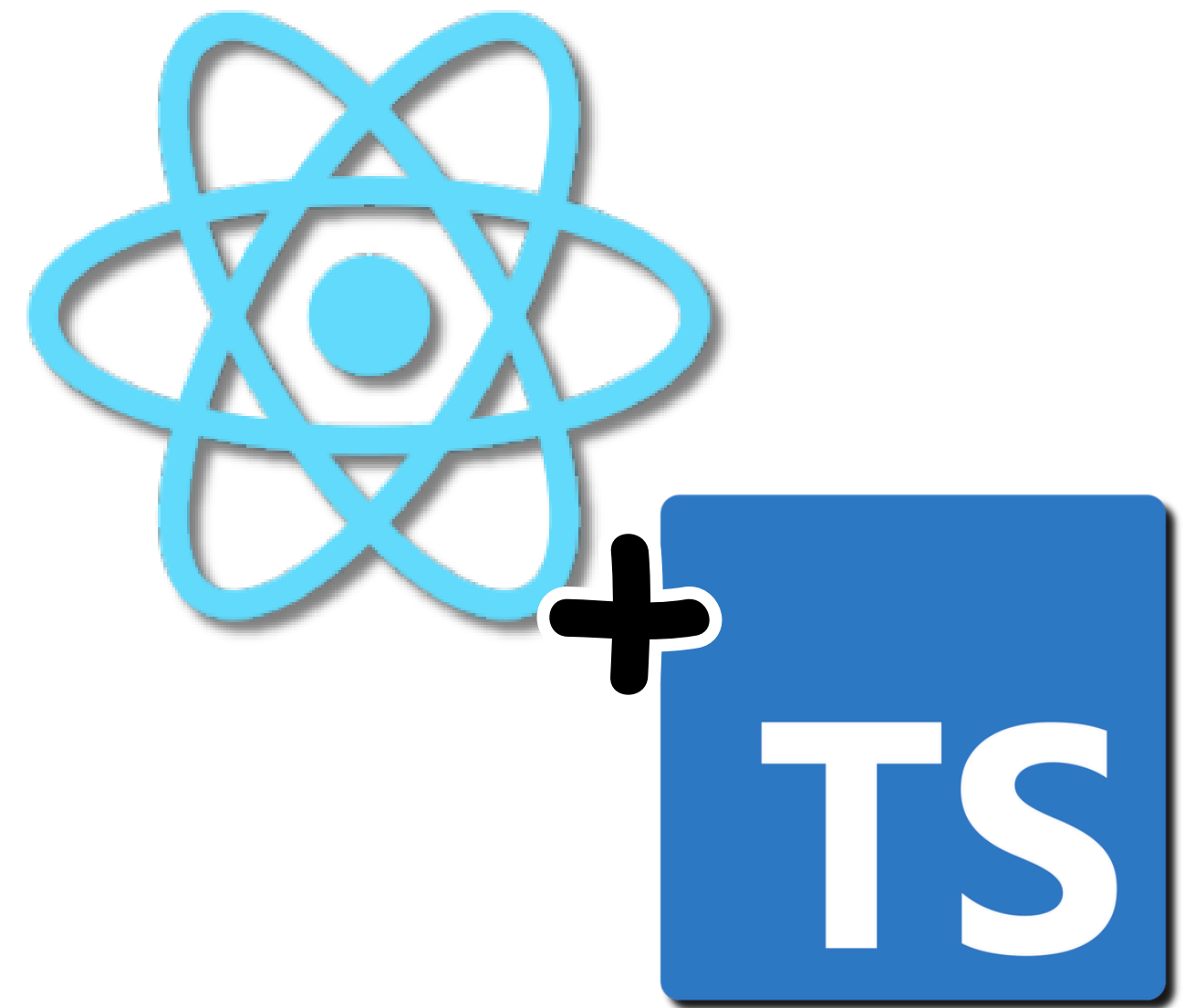


Vite: il build tool moderno

- Avvio istantaneo grazie all'uso di ESM nativi
- Hot Module Replacement (HMR) ultra-veloce
- Ottimizzazione automatica per la produzione
- Configurazione minimale

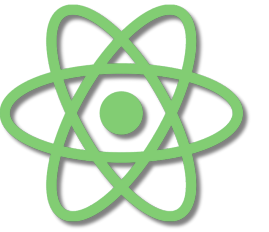
TypeScript integrato

- Type checking durante lo sviluppo
- Autocomplete intelligente nell'IDE
- Prevenzione di errori comuni
- Documentazione implicita del codice

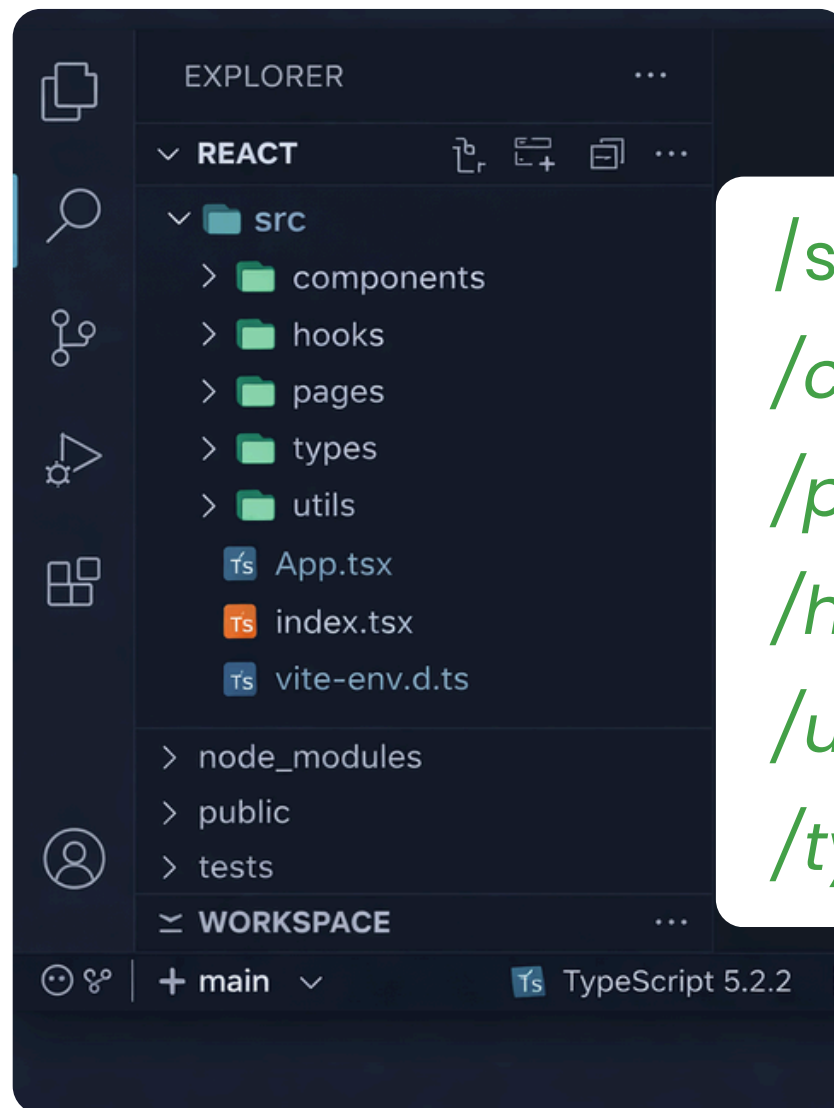


Setup e Introduzione

Struttura di un Progetto React Moderno



Organizzazione tipica:



/src: codice sorgente dell'applicazione
/components: componenti riutilizzabili
/pages o /routes: componenti di pagina
/hooks: custom hooks
/utils: funzioni di utilità
/types: definizioni TypeScript

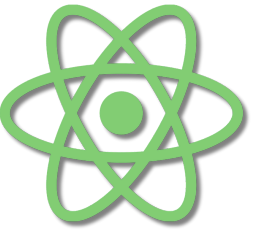
Best practices:

- Separazione delle responsabilità
- Modularità e riusabilità
- Convenzioni di naming chiare



Setup e Introduzione

Node.js e Package Manager



Node.js:

- Runtime JavaScript lato server
- Permette di eseguire tool di sviluppo
- Ecosistema NPM per librerie e dipendenze

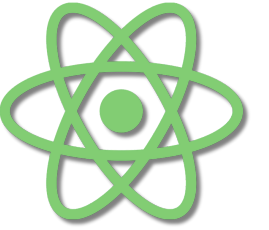
Package Manager:

- **npm**: il più diffuso, incluso con Node.js
- **yarn**: veloce e deterministico
- **pnpm**: efficiente nello spazio disco, usa hard links

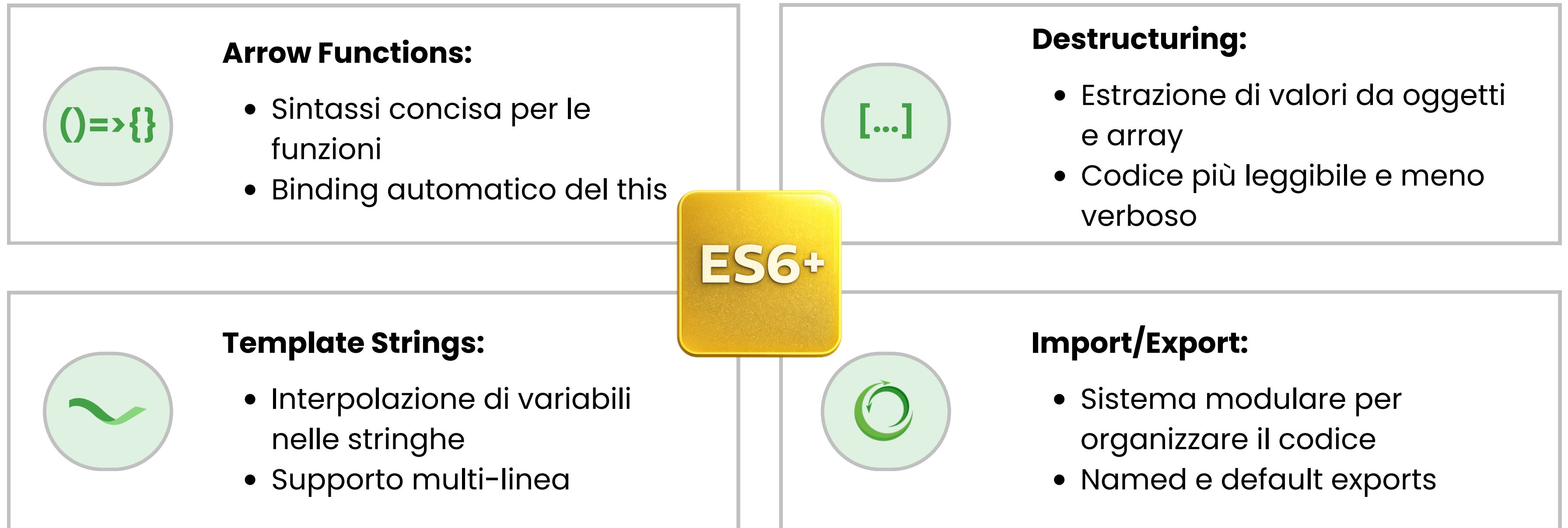
Utilizzo:

- Installazione dipendenze
- Script di sviluppo e build
- Gestione versioni delle librerie

Setup e Introduzione

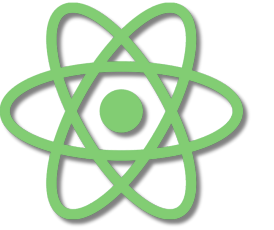


Ripasso ES6+: Modern JavaScript



Setup e Introduzione

Moduli: Import ed Export



Named Exports:

- Esportare multiple entità da un modulo
- Import con nome specifico

Default Export:

- Un export principale per modulo
- Import con nome personalizzabile

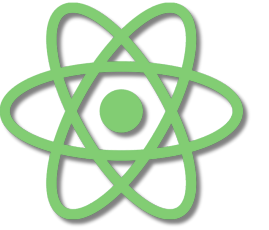
Vantaggi:

- Riusabilità del codice
- Separazione delle responsabilità
- Tree-shaking automatico (rimozione codice inutilizzato)
- Namespace isolation



Setup e Introduzione

Tooling e DevTools



React DevTools:

inspect componenti,
props, state



ESLint: linting per
codice consistente



Prettier:

formattazione
automatica



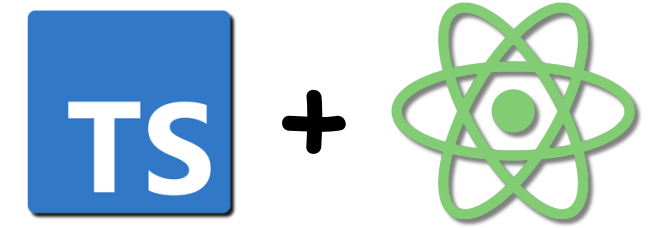
Vite DevTools: analisi
bundle e performance

Browser DevTools:

- Console per debugging
- Network tab per richieste HTTP
- Performance profiler
- React Profiler per ottimizzazioni

TypeScript

Perché TypeScript con React



Vantaggi principali:

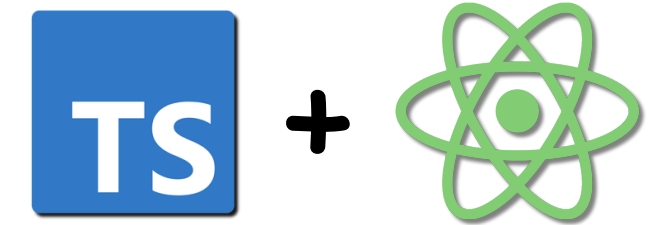
- **Type Safety:** errori individuati in fase di sviluppo
- **IntelliSense:** autocomplete e documentazione inline
- **Refactoring:** modifiche sicure e assistite
- **Scalabilità:** codice più mantenibile nei progetti grandi

TypeScript + React:

- Props e state tipizzati
- Eventi con tipi corretti
- Componenti auto-documentanti
- Riduzione di bug in produzione

TypeScript

Tipi Base in TypeScript



Tipi primitivi:



string: testo



number: numeri (interi e decimali)



boolean: true/false



null e undefined



any: tipo dinamico (evitare)

Tipi composti:

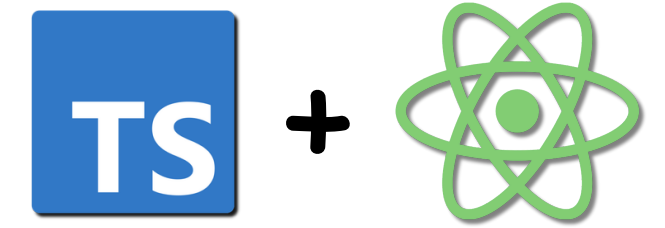
array: number[] o Array<number>

tuple: [string, number]

object: strutture complesse

Inference: TypeScript deduce i tipi automaticamente quando possibile

TypeScript



Interfacce e Tipi Personalizzati

Interface:

- Definisce la struttura di un oggetto
- Estendibile e componibile
- Ideale per oggetti e classi

Type Alias:

- Definisce qualsiasi tipo personalizzato
- Union types, intersection, ecc.
- Più flessibile delle interfacce

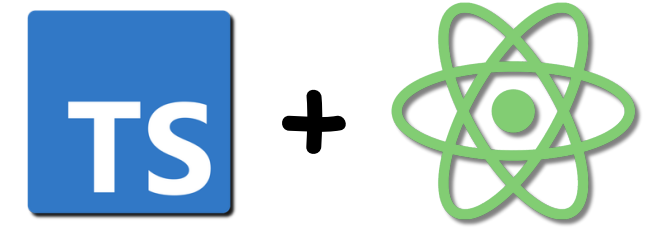


Quando usare cosa:

- **Interface** per oggetti che potrebbero essere estesi
- **Type** per union, tuple, primitive composte
- **Entrambi** validi per definire props dei componenti

TypeScript

TSX: TypeScript + JSX



JSX (JavaScript XML):

- Sintassi per descrivere l'UI in modo dichiarativo
- Sembra HTML ma è JavaScript

TSX:

- JSX con type checking di TypeScript
- Props verificate staticamente
- Eventi con tipi corretti

```
1
2 <div>
3   <ul>
4     <li><input type="checkbox" key="1" id="checkbox-1" /> string</li>
5   </li>
6     <li><input type="checkbox" key="2" id="checkbox-2" checked />
7   </li>number
8     <li><input type="checkbox" key="3" id="checkbox-3" /> boolean</li>
9   </li>
10 </div>
10 </div>
```

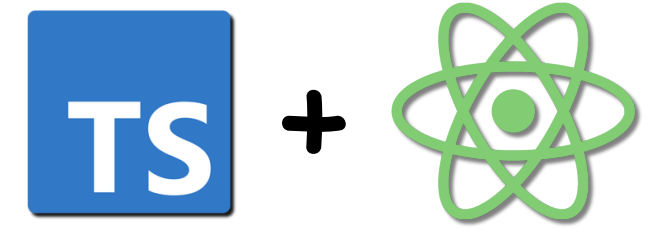
Vantaggi

- Errori di battitura nelle props rilevati subito
- Autocomplete per props e attributi
- Sicurezza nei tipi degli eventi



TypeScript

Generics in TypeScript



Cosa sono:

- Tipi parametrici riutilizzabili
- Flessibilità mantenendo type safety
- Comuni nelle librerie React

Esempi in React:

- `useState<Type>()`: tipo dello stato
- `Array<Type>`: array tipizzati
- Props generiche per componenti riutilizzabili

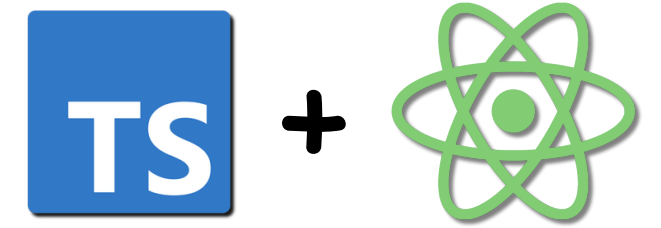
Vantaggi

- Componenti type-safe e flessibili
- Riutilizzabilità senza perdere tipi
- API library ben tipizzate



TypeScript

Utility Types per React



Built-in utility types:

Partial<T>:

tutti i campi
opzionali

Required<T>:

tutti i campi
obbligatori

Pick<T, K>:

seleziona proprietà
specifiche

Omit<T, K>:

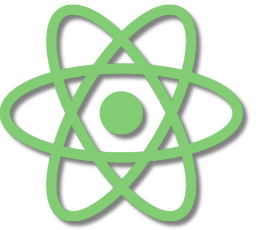
esclude proprietà
specifiche

Utilizzo con React:

- Props opzionali con default values
- Estensione di props esistenti
- Type narrowing e discriminated unions

Componenti

Componenti Funzione in React



Componente:

- Funzione che restituisce JSX
- Blocco riutilizzabile dell'interfaccia
- Può ricevere dati tramite props

Caratteristiche:

- Nome con lettera maiuscola
- Ritorna un solo elemento root (o Fragment)
- Possono essere composti e annidati

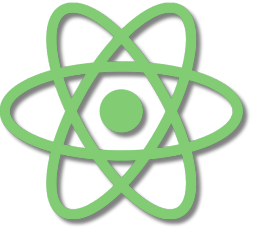
Filosofia:



- UI = funzione dello stato
- Componenti come mattoncini LEGO
- Riusabilità e manutenibilità

Componenti

Props Tipizzate con TypeScript



Props:

- Parametri passati ai componenti
- Rendono i componenti riutilizzabili e configurabili
- Flusso unidirezionale: dal parent al child

Tipizzazione:

- Definire un'interface o type per le props
- Type checking automatico
- Documentazione integrata



Best practices:

- Props immutabili: non modificarle nel componente
- Destructuring per leggibilità
- Default props quando appropriato

Componenti

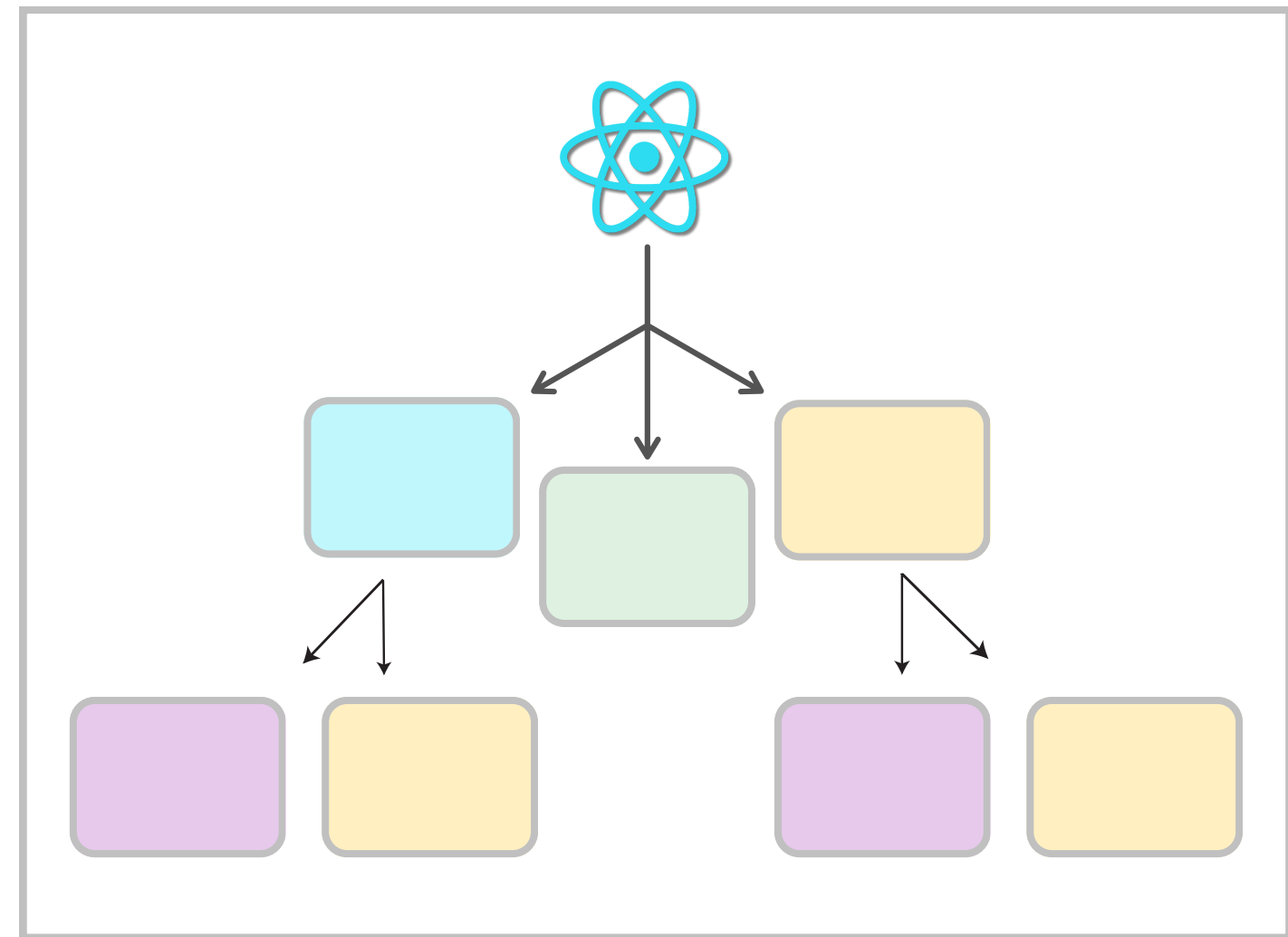
Children e Composizione

Children:

- Props speciale che rappresenta il contenuto interno
- Permette di creare componenti container
- Layout e wrapper components

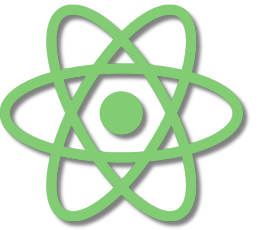
Composizione:

- Costruire UI complesse da componenti semplici
- Alternativa all'ereditarietà
- Maggiore flessibilità e riusabilità



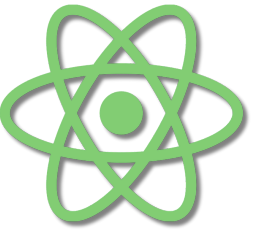
Pattern comuni:

- Layout components (Card, Modal, Container)
- Slot pattern: props per diverse aree del componente
- Render props e compound components



Componenti

Fragment e Key



Fragment:

- Raggruppare elementi senza aggiungere DOM nodes
- Sintassi breve: `<>...</>`
- Sintassi estesa con key:
`<Fragment key={id}>...</Fragment>`

Key prop:

- Identificatore univoco per elementi in liste
- Aiuta React a ottimizzare re-render
- Non usare index come key quando l'ordine cambia

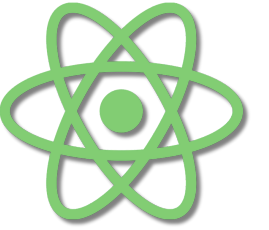


Best practices:

- Usare ID stabili e univoci
- Key a livello del componente mappato
- Evitare chiavi generate random

Componenti

Conditional Rendering



Tecniche principali:

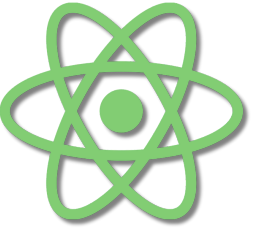
- Operatore ternario: `condition ? <A /> : <B />`
- AND logico: `condition && <Component />`
- If/else tradizionale con return multipli
- Switch/case per condizioni complesse

Pattern comuni:

- Loading states
- Error boundaries
- Empty states e placeholder
- Feature flags e A/B testing

Type safety:

- TypeScript verifica tutte le branch
- Union types per stati mutualmente esclusivi



Componenti

Liste e Rendering con map()

Rendering di array:

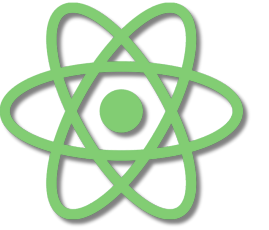
- Metodo `map()` per trasformare array in JSX
- Ogni elemento deve avere una `key` univoca
- Filtrare con `filter()` prima di mappare

Pattern comune:

```
items.map(item => <Component key={item.id} data={item} />)
```

Componenti

Liste e Rendering con `map()`



Best practices:

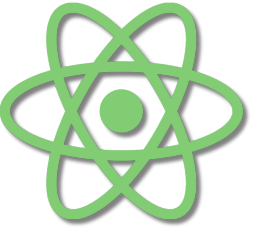
- Usare ID stabili e univoci
- Key a livello del componente mappato
- Evitare chiavi generate random

Operazioni comuni:

- `map()` : trasformare elementi
- `filter()` : selezionare sottoinsiemi
- `sort()` : ordinare prima del render
- `reduce()` : aggregare dati

Stato e Eventi

Introduzione a useState



Stato:

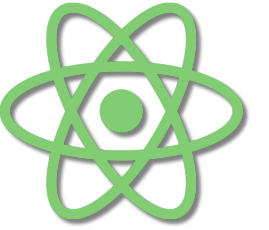
- Dati che cambiano nel tempo
- Causano re-render quando modificati
- Locali al componente

useState Hook:

- Aggiunge stato ai componenti funzione
- Ritorna valore corrente e funzione setter
- Preserva lo stato tra i re-render

Principi:

- Stato immutabile: non modificare direttamente
- Aggiornamenti asincroni: batch updates
- Initializer function per valori costosi



Stato e Eventi

State Updates e Batching

Aggiornamenti multipli:

- React raggruppa più setState
- Un solo re-render per batch
- Miglioramento performance automatico

Functional updates:

- Uso della funzione updater:
`setState(prev => prev + 1)`
- Necessario quando dipendi dallo stato precedente
- Garantisce valore sempre aggiornato

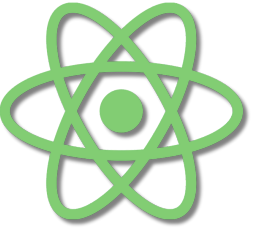
```
state = {
  dogId: 991,
  dogName: 'Pluto',
  dogAge: 8,
  dogIsMicroChipped: true,
  dogLastVaccineDate: 1538784000,
  dogNeedsVaccination: false,
}

this.setState({
  dogNeedsVaccination: true,
});
```

Stato derivato:

- Calcolare valori durante il render
- Non duplicare dati nello stato
- Usare useMemo per computazioni costose

Stato e Eventi



Gestione degli Eventi in React



Eventi sintetici:

- React normalizza gli eventi tra browser
- Stessa API cross-browser
- Delegazione automatica per performance

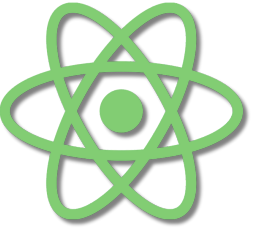
Handler pattern:

- Funzioni passate come props agli elementi
- Convenzione `onNomeEvento`
- Binding automatico con arrow functions

Eventi comuni:

- Click, change, submit, input
- Keyboard, mouse, focus events
- Prevenzione comportamento default

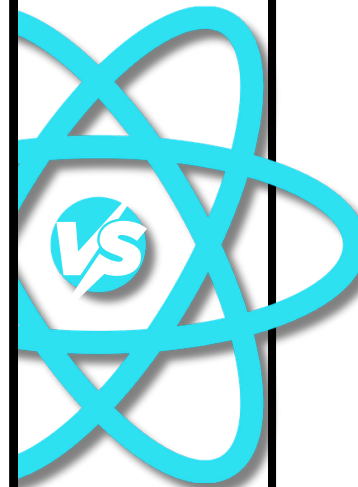
Stato e Eventi



Props vs State: Differenze e Interazione

Props:

- Dati ricevuti dal parent
- Immutabili nel componente child
- Configurazione del componente



State

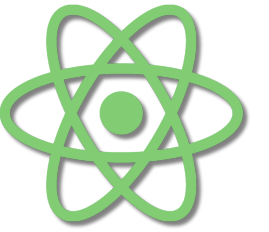
- Dati interni al componente
- Mutabili tramite setter
- Causano re-render quando cambiano

Interazione:

- **Lifting state up:** stato condiviso nel parent
- **Props callback:** child comunica con parent
- **Single source of truth:** uno stato, molte props

Stato e Eventi

Custom Hooks



Cosa sono:

- Funzioni che incapsulano logica con hooks
- Riutilizzabilità della logica stateful
- Naming convention: **use** prefix

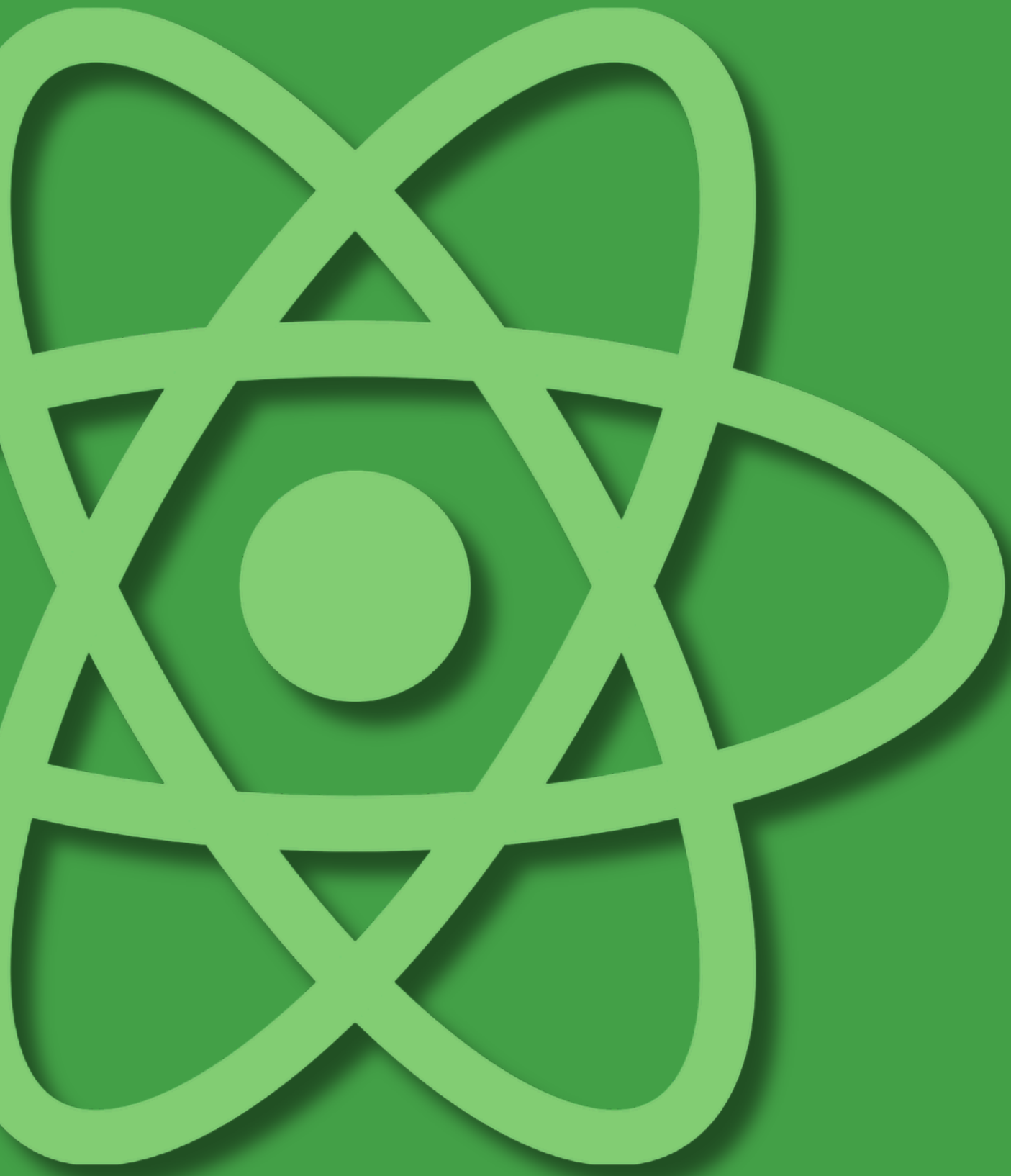
Pattern comuni:

- **useLocalStorage** persistenza dati
- **useDebounce** delay di input
- **useFetch** data fetching riutilizzabile
- **useMediaQuery** responsive logic

Vantaggi:

- Separazione delle responsabilità
- Testing più semplice
- Composizione di comportamenti





Grazie